# [CSED233-01] Data Structure

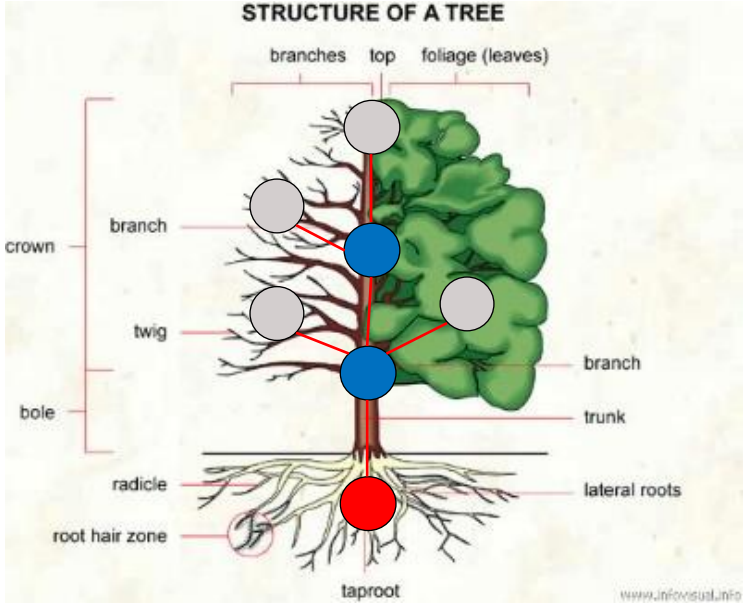
# Tree (1)

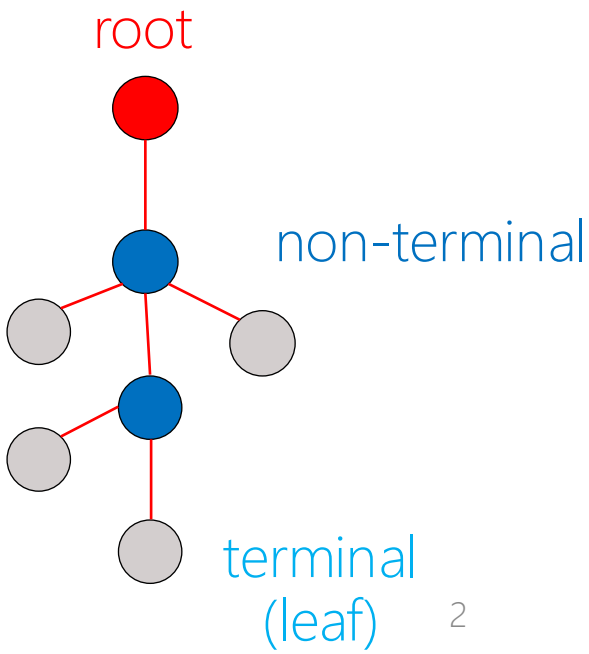
Seung-Hwan Baek

POSTECH

# Tree

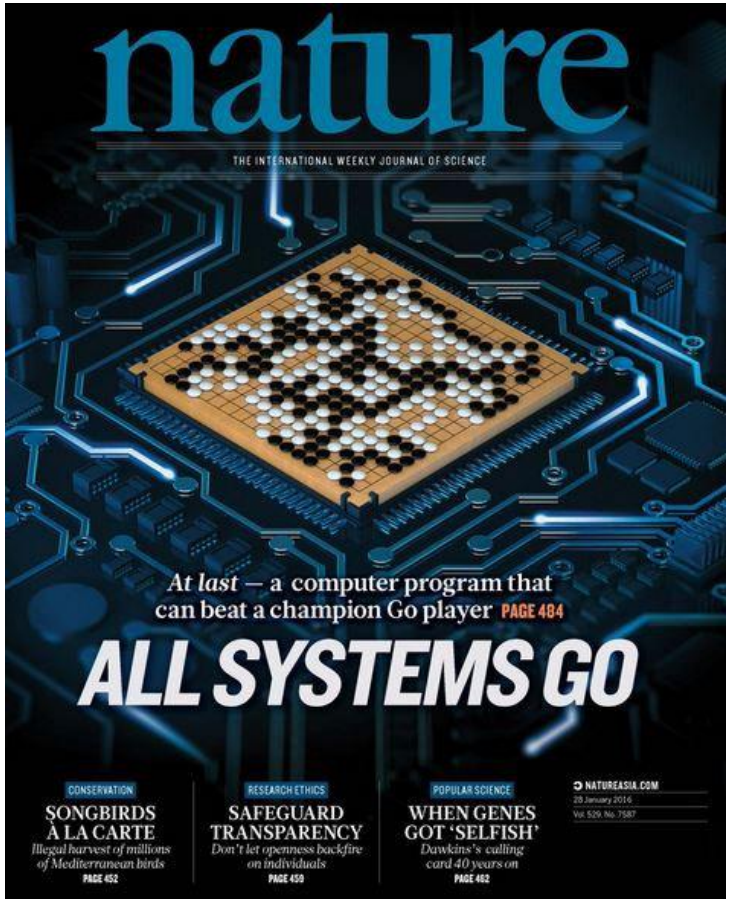
- Tree is a collection of nodes (depicted as circles) and edges (depicted as lines)
- Tree has a node distinguished as a "root"
- One edge in a tree connects two nodes: child and parent
- One node in a tree can be connected to many edges
- Flip the normal tree upside down so that root can be at the top

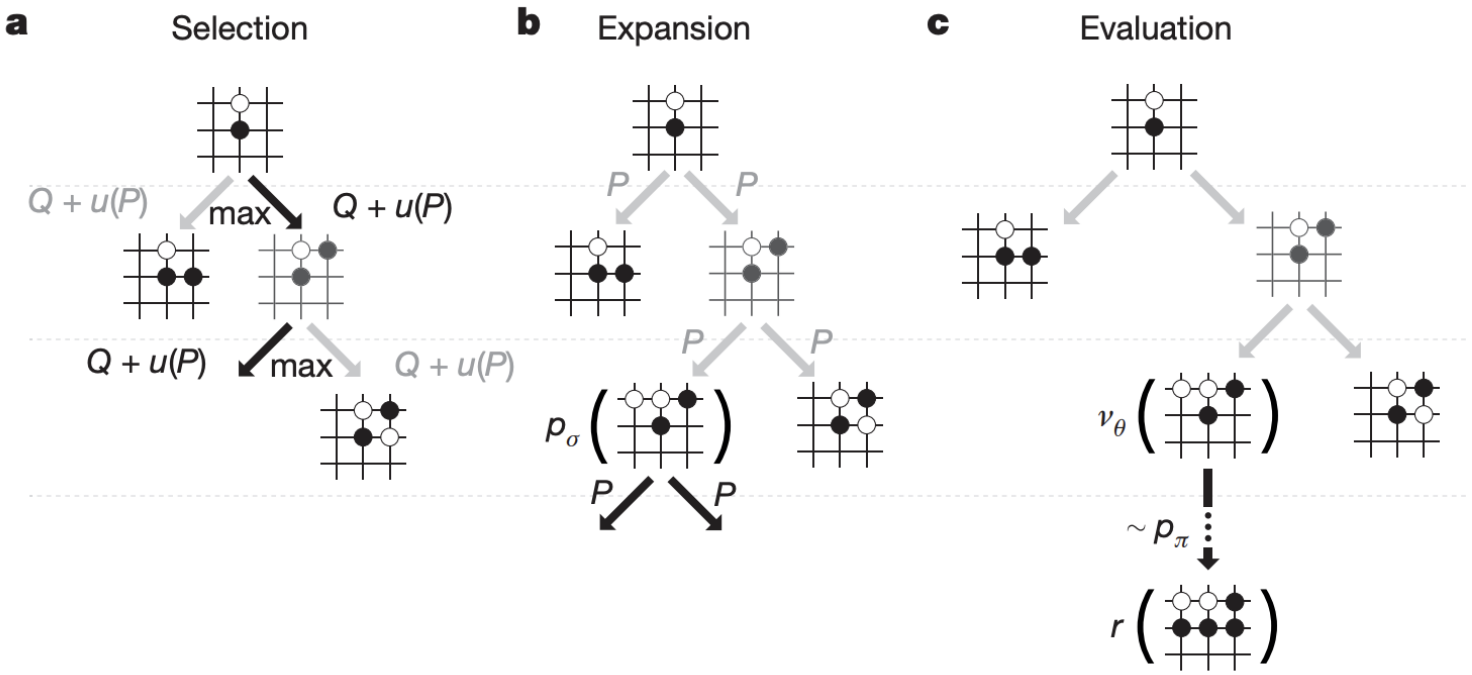


# Quiz: Tree Example

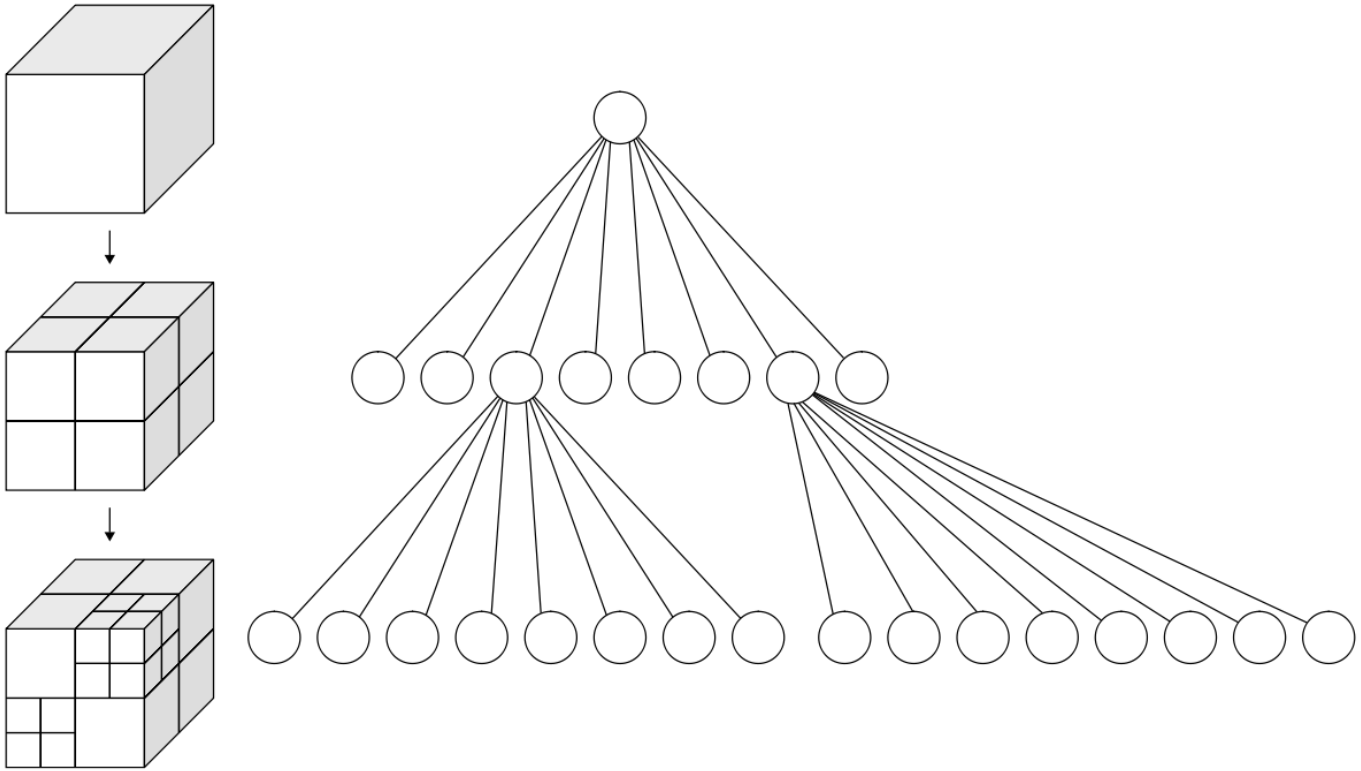
- What does the node and edge mean here?

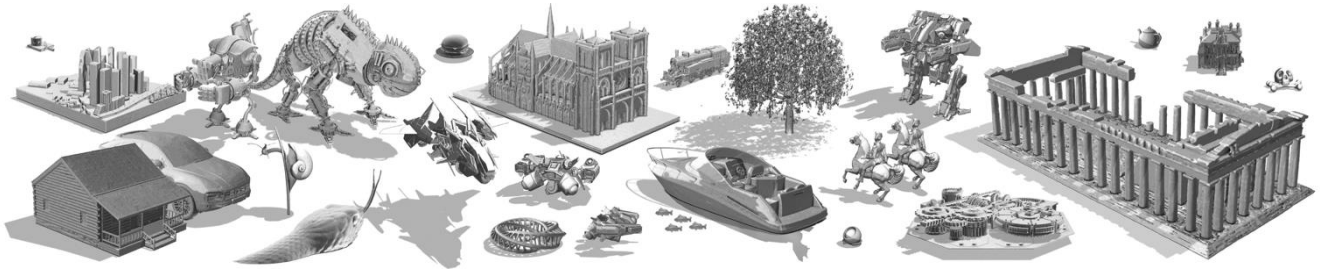


# Quiz: Tree Example

- What does the node and edge mean here?



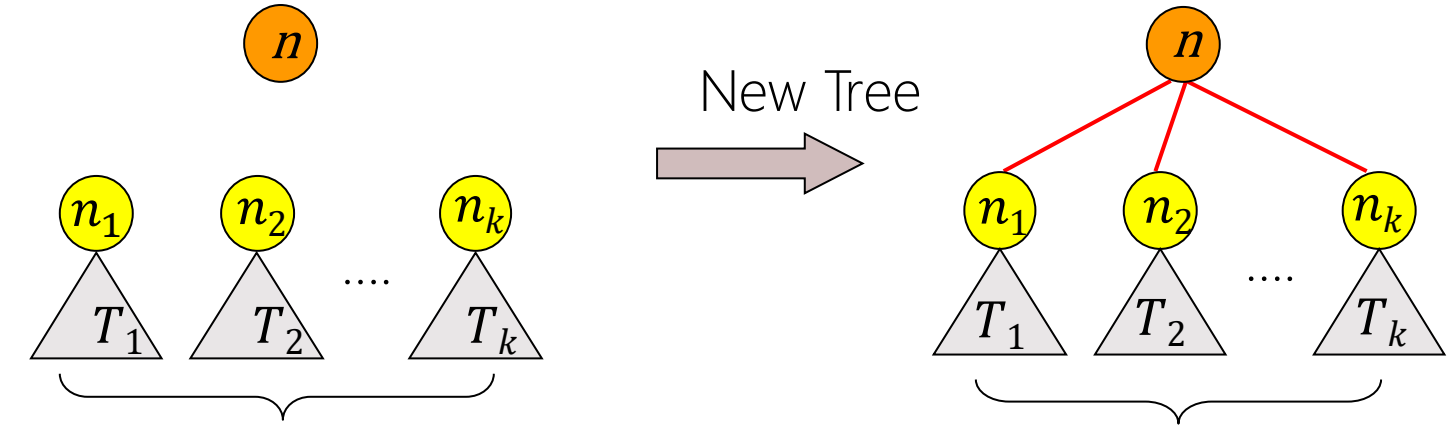


# Recursive Definition

- Starts with a single node

Single node

- Build a new tree by making other trees as subtrees of the node




Trees with roots $n_1, n_2, \ldots, n_k$

Subtrees

# Parenthood Relations

- Parent/child

$A$ is the parent of $C$

$C$ is the child of $A$

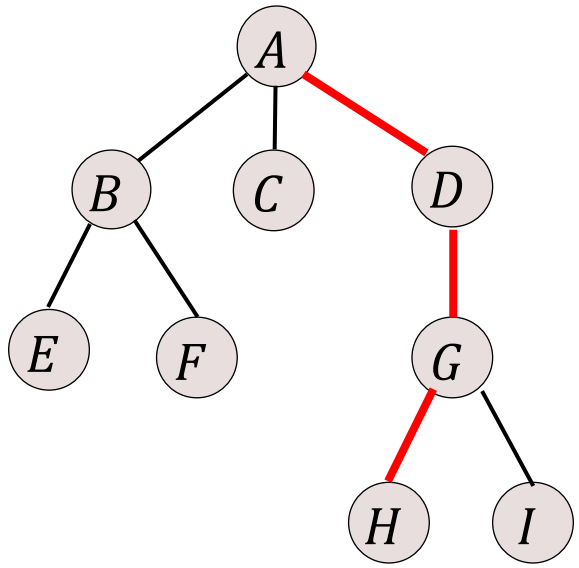
- Ancestor/descendant

$A$ is the ancestor of $G$

$G$ is the descendant of $A$

- Siblings

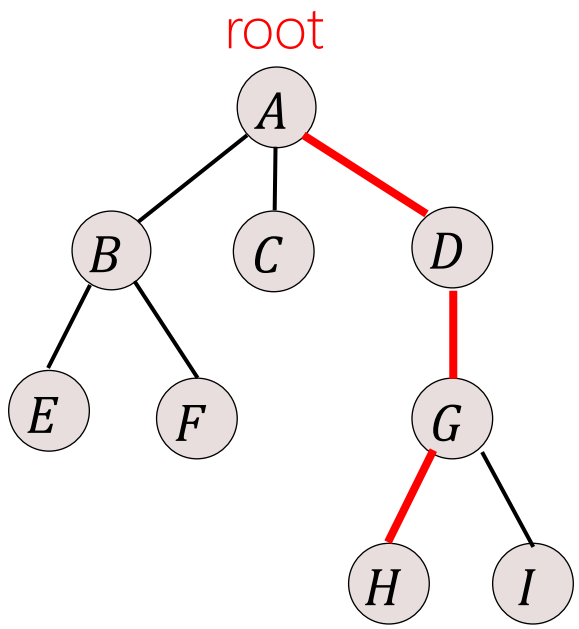
$B, C, D$ are siblings



# Path and Leaf Nodes

- Path $< n_1, n_2, \ldots, n_k >$ ?
  - $n_i$ is the parent of $n_{i+1}$ $(1 \quad i < k)$
  - $length = k - 1$
    - Number of edges connecting the path

- Terminal (leaf, external) $\leftrightarrow$ Non-terminal (internal)
  - Whether the node has any child nodes
  - $E, F, C, H, I$ ➔ leaf
  - $B, A, D, G$ ➔ Non-terminal

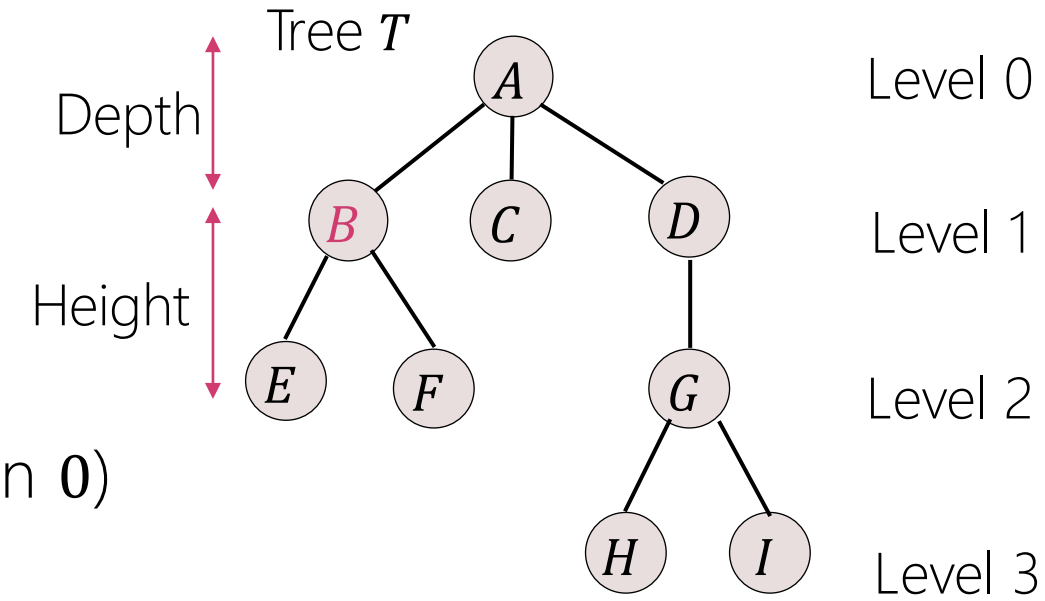



*Path from* $A$ *to* $H$

$(length = 3)$

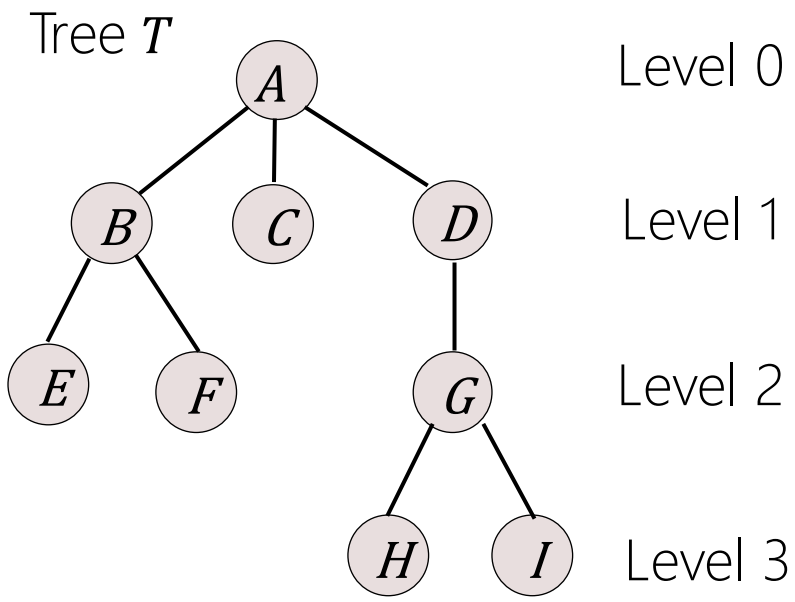
# Depth, Height, Level

- Depth of node $M$
  - the length of the path from the root to $M$
  - the # of ancestors of $M$ (excluding $M$ itself)
- Height of node $M$
  - the length of a longest path from $M$ to a leaf
- Height of the tree= height of the root
- Level
  - Root is at level 0
  - Its children are at level 1
  - Their children are level 2, and so on
  - Level number could start at $1$ (rather than $0$)
  - Again, no rule here



# Examples

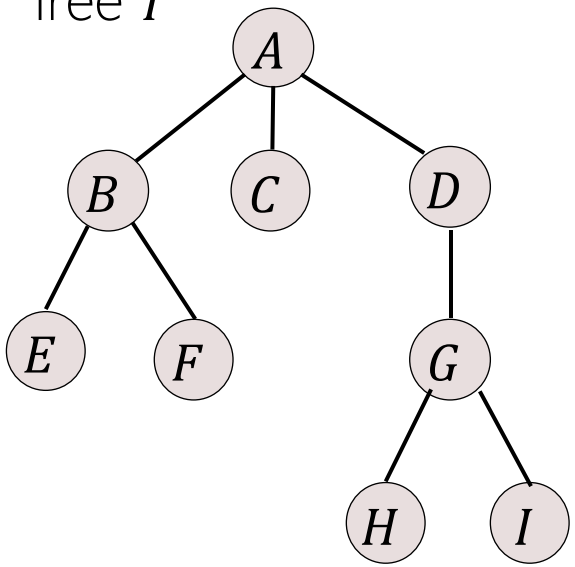
- $Depth\ (node\ G\ ) = 2$
- $Height\ (node\ G\ ) = 1$
- $Height\ (node\ F\ ) = 0$
- $Height\ (tree\ T\ ) = 3$



# Degree

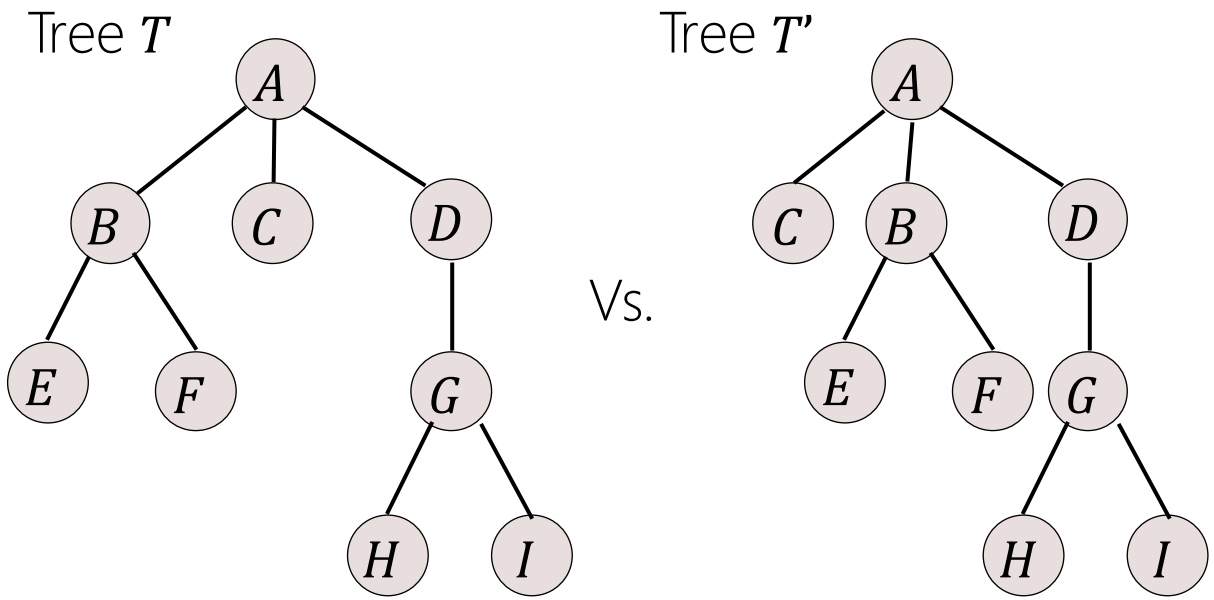
- Degree of node
  - # of children
  - $Degree\ (leaf)\ =\ 0$
- Degree of tree
  - maximum of its node degrees

Tree $T$



# Ordered

- If there exists a linear ordering between siblings
- It is an ordered tree



# Forest

- A collection/set of trees

# Binary Trees

- Binary tree
  - Every node has at most two children
  - Each child is designated as a left child or a right child

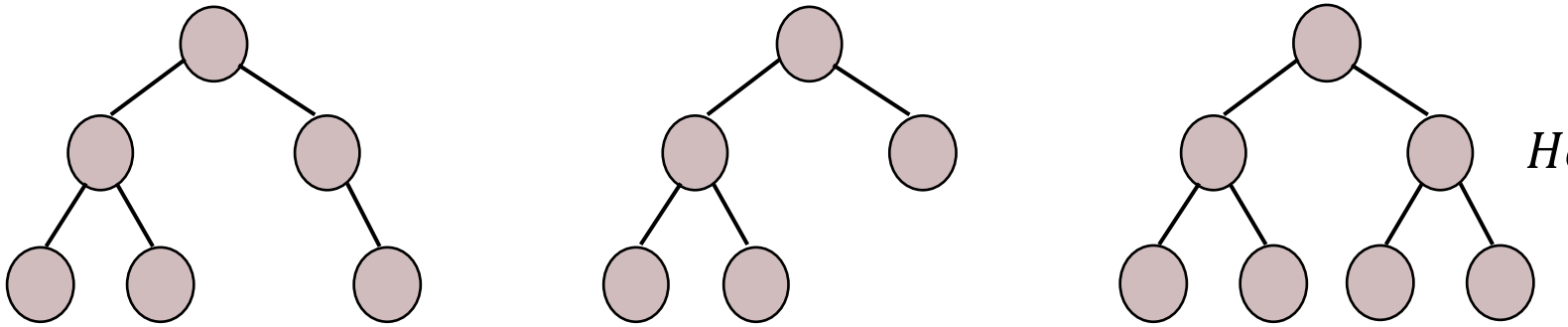
- Examples:

# Quiz

- Binary tree is an ordered tree?

# Proper, Full Binary Trees

- Proper
  - if each node has either zero or two children
- Full
  - If it has a maximum # of nodes at each level
  - A full binary tree of height $h$ has $(2^{h+1} - 1)$ nodes



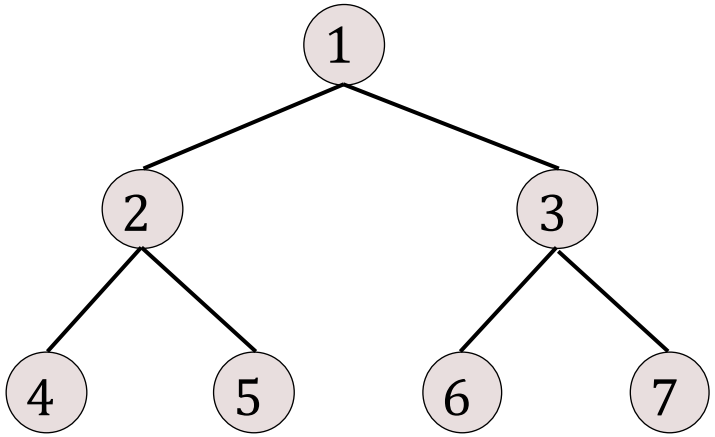
proper (not full) full

$Height = 2$

- Caution: *different definitions* in some texts

# Node Numbering

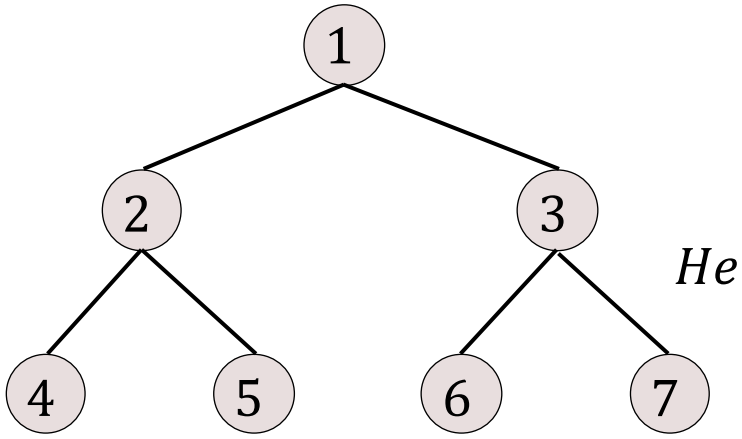
- How to assign numbers to nodes (in a full binary tree)?
- Number the nodes $1$ through $2^{h+1} - 1$
- Number by levels from top to bottom
- Within a level, number from left to right




$Height = 2$

# Node Numbering

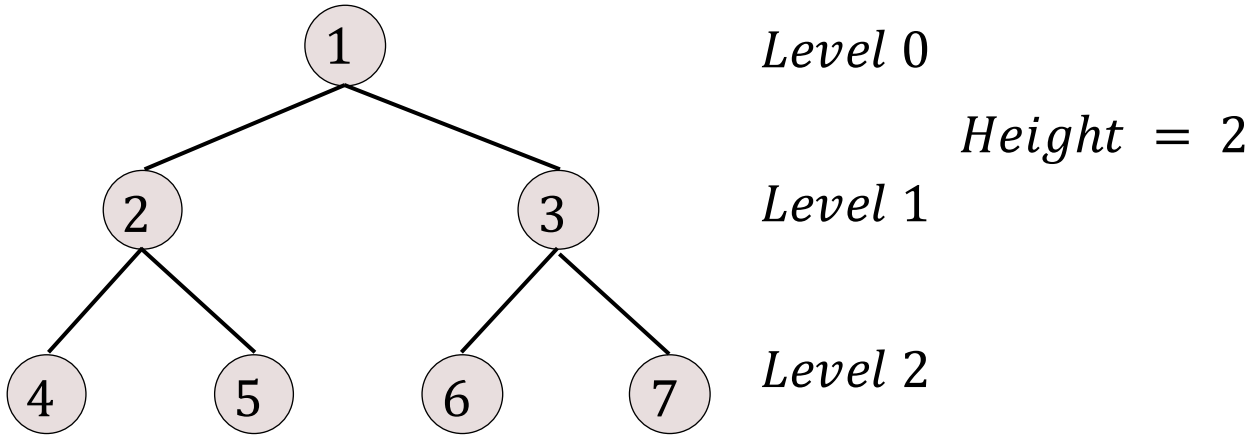
- Properties of node numbers
  - Node $1$ is the root
  - $Parent$ (node $k$)
    - node $floor(k/2)$(if $k \neq 1$)
  - $Left_child$ (node $k$)
    - node $2k$ (if $2k < n$)
  - $Right_child$ (node $k$)
    - node $2k + 1$ (if $2i + 1 < n$)
  - $Left_sibling$ (node $k$)
    - node $k - 1$ (if $k$ is odd)
  - $Right_sibling$ (node $k$)
    - node $k + 1$ (if $k$ is even and $k + 1 < n$)



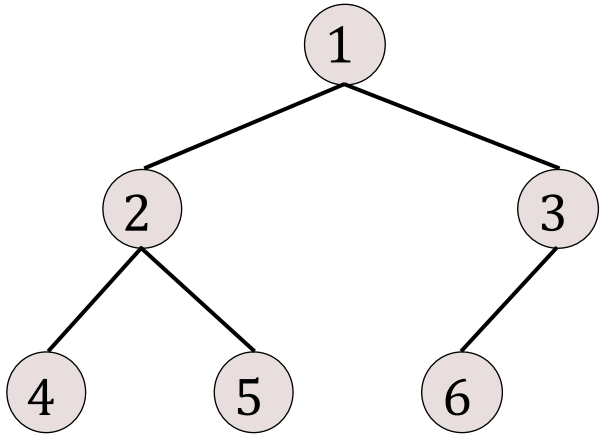

$Height = 2$

# Complete Binary Tree

- Relaxed definition of a full binary tree
- A binary tree of height $h$ is complete, if
  - All levels (possibly except $h$) are completely full
  - Level $h$ (leaf level) is filled from left to right




Full binary tree with 7 nodes




Complete binary tree with 6 nodes

# Binary Tree Implementations

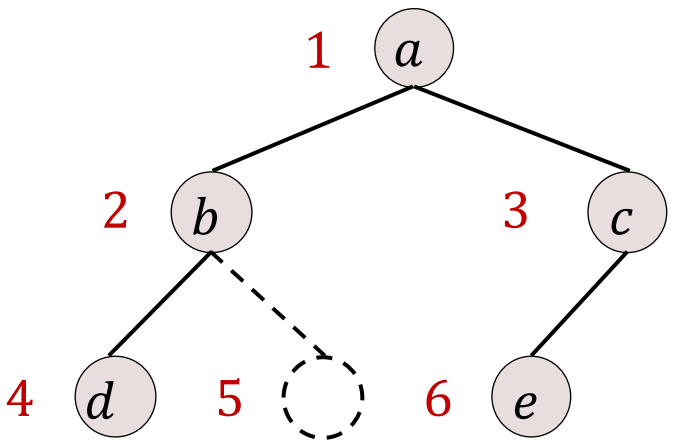
- Array implementation
  - Elements for a complete binary tree
  - $char\ binary_tree[123];$

- Linked implementation
  - Each node has two pointer fields

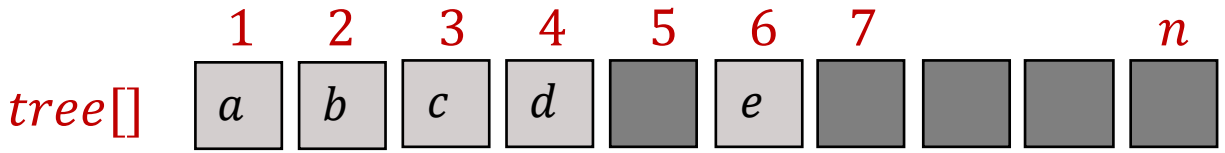
$$
\begin{aligned}
&struct\ elem\{\\
&\quad char\ val;\\
&\quad elem * \ left;\\
&\quad elem * \ right;\\
&\}
\end{aligned}
$$

# Array Implementation: Binary

- Using the numbering scheme for a full binary tree
- The node numbered $k$ is stored in an array $tree[k]$

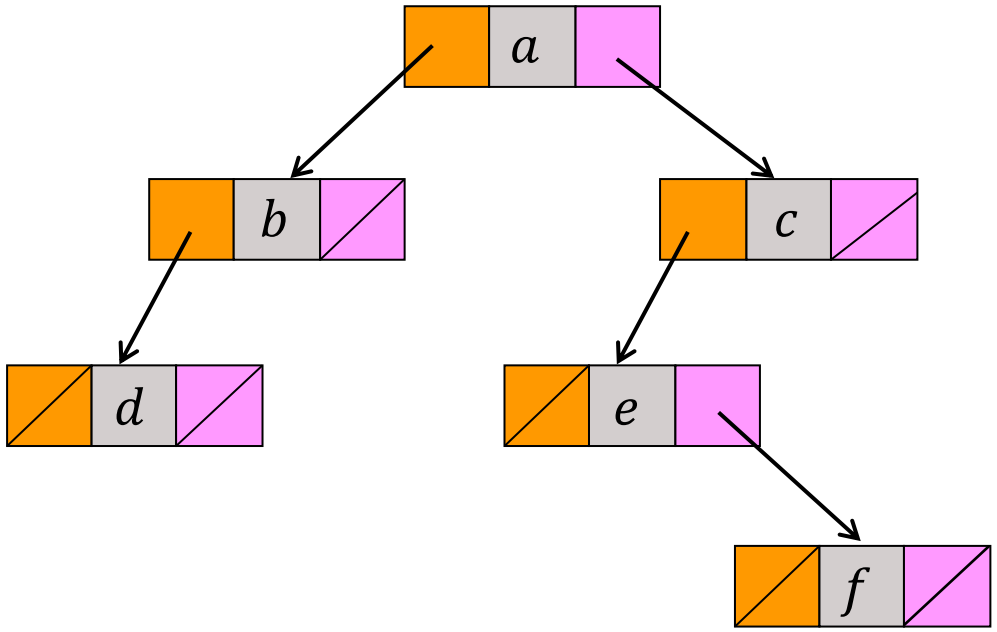



Binary tree with $6$ nodes and $1$ missing one



# Linked Implementation: Binary

- Each binary tree node has
  - Value field
  - Two pointer fields (for left & right children)



# Quiz

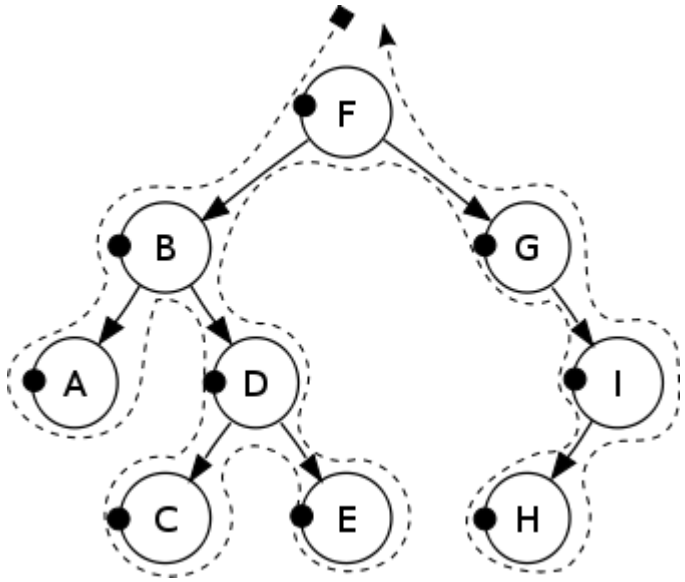
- What are the pros and cons of the array implementation and the linked implementation?

# Tree Traversal

- Passing through the tree & visit each of its nodes
- Visiting each tree node exactly once in a systematic way
- During the visit, actions are taken
  - Print, update, check, evaluate, ....
- Example traversal

$$F\ B\ A\ D\ C\ E\ G\ I\ H \text{ (visiting } \textit{once}\text{)}$$

- Tree traversal is a way of linearizing the tree



# Tree Traversal

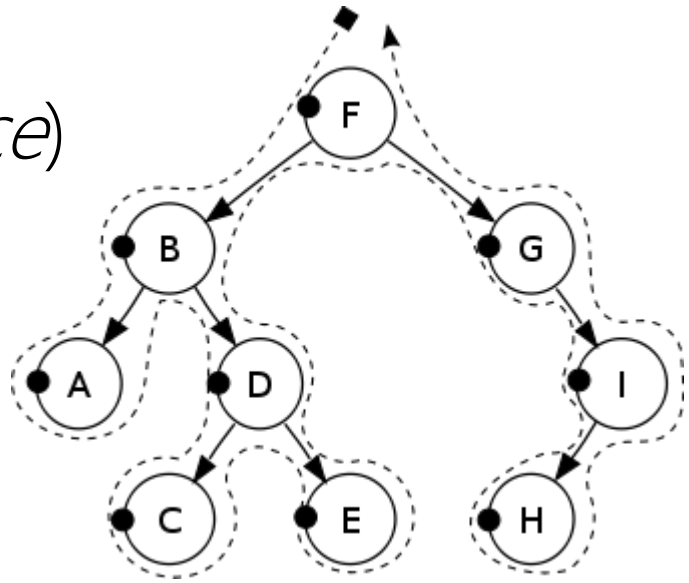
- Example traversal

$$F\ B\ A\ D\ C\ E\ G\ I\ H \text{ (visiting } \textit{once}\text{)}$$

- This is called depth-first search (DFS)
- Pseudo code for this DFS

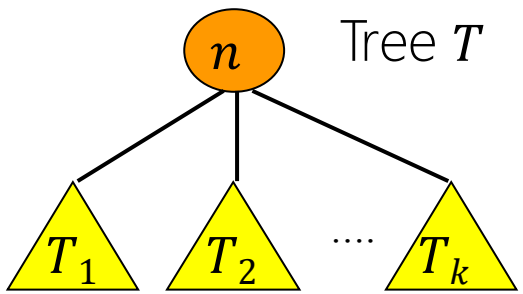
```
func traverse(node){
   if the node have a left child
      call traverse for the left child;


   if the node have a right child
      call traverse for the right child;
}
```



# Types of Traversals

- Action usually happens at a position among three options
  - preorder, inorder, and postorder




```
Func traverse (node){
  Action on the node
  if the node have a left child
    call traverse for the left child;
  Action on the node
  if the node have a right child
    call traverse for the right child;
  Action on the node
}
```

Preorder → Action on the node

Inorder → Action on the node

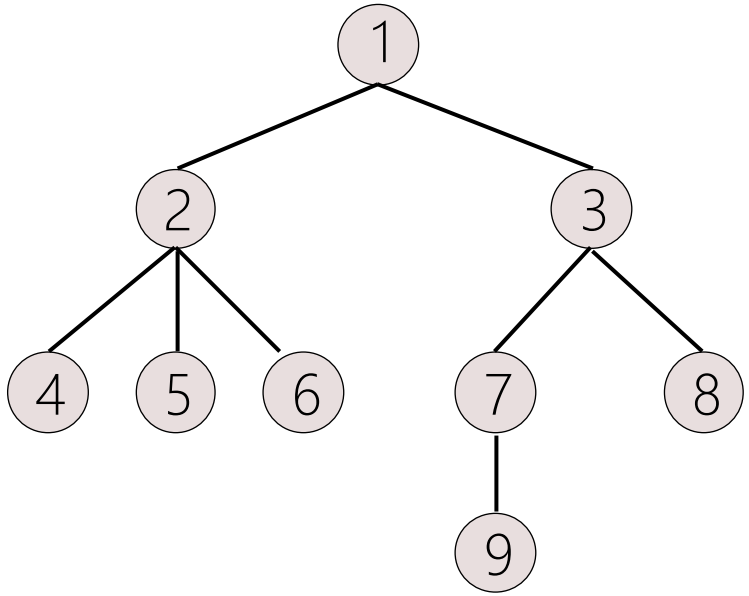
Postorder → Action on the node

- $Preorder(T) = < n, Preorder(T_1), \ldots, Preorder(T_k) >$
- $Postorder(T) = < Postorder(T_1), \ldots, Postorder(T_k), n >$
- $Inorder(T) = < Inorder(T_1), n, Inorder(T_2), \ldots, Inorder(T_k) >$,

, …

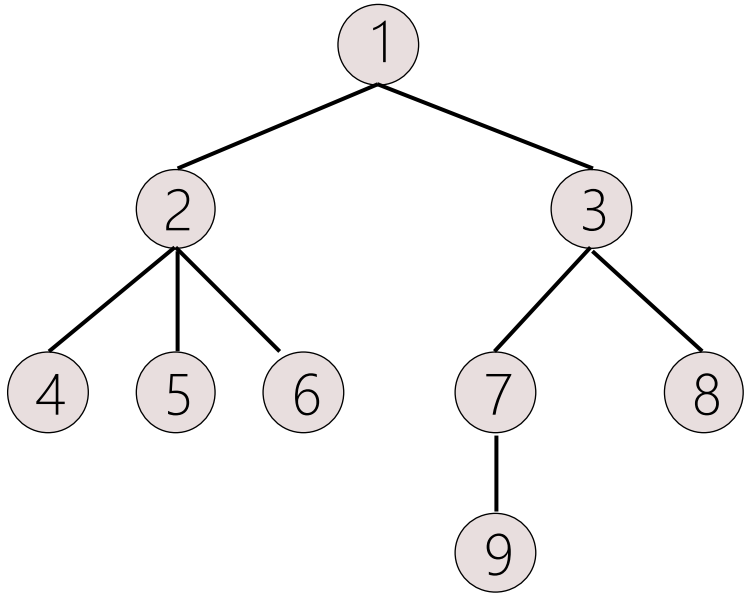
# Quiz

- $Preorder(T)$
  $=$



# Quiz

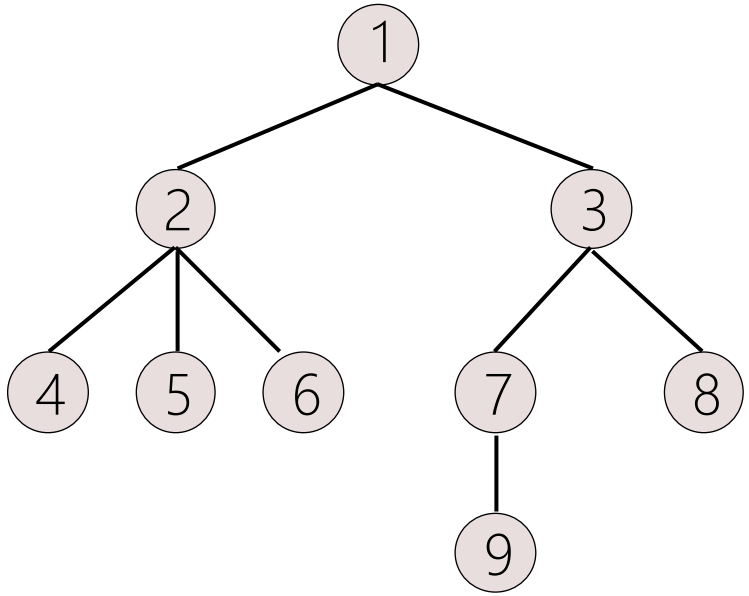
- $Postorder(T)$
  $=$



# Quiz

- $Inorder(T)$

  $=$



# Quiz

- Consider a tree where every internal node has exactly one child
- For this tree structure, what relationship holds among preorder, inorder, and postorder traversal results?
  - Preorder vs. Postorder
  - Inorder vs. Preorder
  - Inorder vs. Postorder
- If all children are left children
- If all children are right children